A2 ACTL3162

Nikola Drpa

2026-10-29

```r
### Packages ###


### Global Parameters ###
c_0 <- 55
theta <- 0.08
reins_loading <- 0.12
horizon <- 60
prodA_alpha <- 0.80
prodB_d <- 3.889

### Establishing Claim Arrival/Size Distribution ###
claim_scenarios <- data.frame(scenario = c("1", "2", "3", "4"),
                              lambda = c(2.4, 2.4, 4.8, 4.8),
                              shape  = c(1, 0.35, 1, 0.45),
                              rate   = c(0.8, 0.28, 1.6, 0.72))

### Set Seed ###
set.seed(676)
```

```r
### Question 1 - Estimating Ruin Probability ###
# Estimating Function
est_ruin_prob <- function(lambda, shape, rate,
                          c0 = 55, theta = 0.08, horizon = 60, n_sim = 1e5) {
  #Expected Claim Size
  EY   <- shape / rate
  #Premium Rate per Month
  prem <- (1 + theta) * lambda * EY

  ruined <- vapply(seq_len(n_sim), function(i) {
    #Number of Claims
    N <- rpois(1, lambda * horizon)
    if (N == 0) return(FALSE)
    #Claim Arrival Times
    times <- sort(runif(N, 0, horizon))
    Y     <- rgamma(N, shape = shape, rate = rate)
    #Surplus as C(t)
    any(c0 + prem * times - cumsum(Y) < 0)
  }, logical(1))

  p  <- mean(ruined)
  se <- sqrt(p * (1 - p) / n_sim)
```

```r
  c(psi = p, se = se, lower = p - 1.96 * se, upper = p + 1.96 * se)
}

### Q1 Results
ruin_probability <- t(sapply(seq_len(nrow(claim_scenarios)), function(n)
  est_ruin_prob(claim_scenarios$lambda[n], claim_scenarios$shape[n], claim_scenarios$rate[n])))

ruin_probability <- data.frame(scenario = claim_scenarios$scenario, ruin_probability)

# Which Scenario Does not Meet 99% ?
ruin_probability$survival <- 1 - ruin_probability$psi

ruin_probability
```

```
##   scenario     psi           se         lower        upper survival
## 1        1 0.00310 1.757951e-04  2.755442e-03 3.444558e-03  0.99690
## 2        2 0.03326 5.670430e-04  3.214860e-02 3.437140e-02  0.96674
## 3        3 0.00003 1.732025e-05 -3.947687e-06 6.394769e-05  0.99997
## 4        4 0.00105 1.024157e-04  8.492652e-04 1.250735e-03  0.99895
```

```r
### Question 2 - Analysis of Reinsurance Products ###
## a) Cede Same Expected Claim Amount
scenario2 <- claim_scenarios[claim_scenarios$scenario == "2", ]
lambda_sc2 <- scenario2$lambda
shape_sc2 <- scenario2$shape
rate_sc2 <- scenario2$rate
scen2_EY <- shape_sc2 / rate_sc2

# Product A - Proportional Reinsurance
exp_claim_ceded_prodA <- function(alpha, shape = shape_sc2, rate = rate_sc2) {
  EY <- shape / rate
  (1 - alpha) * EY
}

# Product B - Excess of Loss Reinsurance
exp_claim_ceded_prodB <- function(d, shape = shape_sc2, rate = rate_sc2){
  if (is.infinite(d)) return (0)
  EY <- shape / rate
  EY * pgamma(d, shape + 1, rate, lower.tail = FALSE) - d * pgamma(d, shape, rate, lower.tail = FALSE)
}

# Results
amount_ceded_A <- exp_claim_ceded_prodA(prodA_alpha)
amount_ceded_B <- exp_claim_ceded_prodB(prodB_d)

cat("Amount Ceded Under Product A:", amount_ceded_A, "\n")
```

```
## Amount Ceded Under Product A: 0.25
```

```r
cat("Amount Ceded Under Product B:", amount_ceded_B, "\n")
```

```
## Amount Ceded Under Product B: 0.2500192
```

```r
# Reinsurance Premium Rate
reinsurance_prem_rate <- function(type = "none", alpha = 1, d = Inf,
                                  lambda = lambda_sc2, xi = reins_loading) {
  amount_ceded <- switch(type,
                  none = 0,
                  prop_prodA = exp_claim_ceded_prodA(alpha),
                  xl_prodB   = exp_claim_ceded_prodB(d),
                  stop("type must be selected"))
  (1 + xi) * lambda * amount_ceded
}

premium_rates_q2 <- data.frame(
  product         = c("None", "A", "B"),
  reins_prem_rate = c(reinsurance_prem_rate("none"),
                      reinsurance_prem_rate("prop_prodA", alpha = prodA_alpha),
                      reinsurance_prem_rate("xl_prodB",   d = prodB_d)))
premium_rates_q2$net_prem_rate <- (1 + theta) * lambda_sc2 * scen2_EY - premium_rates_q2$reins_prem_rat
print(premium_rates_q2)
```

```
##   product reins_prem_rate net_prem_rate
## 1    None       0.0000000      3.240000
## 2       A       0.6720000      2.568000
## 3       B       0.6720517      2.567948
```

```r
# Updated Ruin Probability Function
est_ruin_prob_reins <- function(type = "none", alpha = 1, d = Inf, lambda = lambda_sc2, shape = shape_s
  EY       <- shape / rate
  # Net Premium
  prem_net <- (1 + loading) * lambda * EY - reinsurance_prem_rate(type, alpha, d, lambda, xi)

  ruined <- vapply(seq_len(n_sim), function(i) {
    # Number of Claims
    N <- rpois(1, lambda * t_max)
    if (N == 0) return(FALSE)
    # Claim Arrival Times
    times <- sort(runif(N, 0, t_max))
    Y     <- rgamma(N, shape = shape, rate = rate)
    # Retained Portion of Each Claim
    Y_ret <- switch(type,
                    none = Y,
                    prop_prodA = alpha * Y,
                    xl_prodB = pmin(Y, d),
                    stop("type must be selected"))
    any(c0 + prem_net * times - cumsum(Y_ret) < 0)
  }, logical(1))

  p  <- mean(ruined)
  se <- sqrt(p * (1 - p) / n_sim)
  c(psi = p, se = se, lower = p - 1.96 * se, upper = p + 1.96 * se)
}

# Ruin Probability - No Reinsurance Products
est_ruin_prod0 <- est_ruin_prob_reins(type = "none")
```

```r
# Ruin Probability - Product A
est_ruin_prodA <- est_ruin_prob_reins(type = "prop_prodA", alpha = prodA_alpha)

# Ruin Probability - Product B
est_ruin_prodB <- est_ruin_prob_reins(type = "xl_prodB", d = prodB_d)

print(est_ruin_prodA)
```

```
##          psi           se        lower        upper
## 0.0126600000 0.0003535495 0.0119670430 0.0133529570
```

```r
print(est_ruin_prodB)
```

```
##          psi           se        lower        upper
## 0.0020600000 0.0001433791 0.0017789770 0.0023410230
```

```r
# Results
ruin_probabilities_q2 <- rbind(est_ruin_prod0, est_ruin_prodA, est_ruin_prodB)
print(ruin_probabilities_q2)
```

```
##                    psi           se       lower       upper
## est_ruin_prod0 0.03269 0.0005623288 0.031587836 0.033792164
## est_ruin_prodA 0.01266 0.0003535495 0.011967043 0.013352957
## est_ruin_prodB 0.00206 0.0001433791 0.001778977 0.002341023
```

```r
## b) Admissible Range of Alpha and d
#Product A
minimum_alpha <- (reins_loading - theta) /  reins_loading

minimum_alpha
```

```
## [1] 0.3333333
```

```r
#Product B
minimum_d <- uniroot(function(dd) reins_loading * exp_claim_ceded_prodB(dd) - theta * scen2_EY, c(1e-6,

minimum_d
```

```
## [1] 0.7946268
```

```r
## c)
# MGFs of the Retained Claim Amount
# No Reinsurance
mgf_none <- mgf_none <- function(r, shape = shape_sc2, rate = rate_sc2) {

  (1 - r / rate)^(-shape)

}
```

```r
# Product A
mgf_prodA <- function(r, alpha, shape = shape_sc2, rate = rate_sc2) {

  (1 - r * alpha / rate)^(-shape)

}

# Product B
mgf_prodB <- function(r, d, shape = shape_sc2, rate = rate_sc2) {

  tail_end_part <- exp(r * d) * pgamma(d, shape, rate, lower.tail = FALSE)

  centre_body_part <- if (r < rate) {
    (rate / (rate - r))^shape * pgamma(d, shape, rate - r)
  } else {
    integrate(function(y) exp(r * y) * dgamma(y, shape, rate), 0, d)$value
  }

  centre_body_part + tail_end_part
}

## Adjusted Coefficients
# No Reinsurance
adj_coef_none <- function(lambda = lambda_sc2, shape = shape_sc2, rate = rate_sc2, loading = theta) {

  prem     <- (1 + loading) * lambda * shape / rate

  lundberg <- function(r) lambda * (mgf_none(r, shape, rate) - 1) - prem * r

  uniroot(lundberg, c(1e-8, rate * (1 - 1e-8)), tol = 1e-12)$root
}

# Product A
adj_coef_prodA <- function(alpha, lambda = lambda_sc2, shape = shape_sc2, rate = rate_sc2, loading = th

  prem_net <- (1 + loading) * lambda * shape / rate -
    reinsurance_prem_rate("prop_prodA", alpha = alpha, lambda = lambda, xi = xi)

  lundberg <- function(r) lambda * (mgf_prodA(r, alpha, shape, rate) - 1) - prem_net * r

  uniroot(lundberg, c(1e-8, (rate / alpha) * (1 - 1e-8)), tol = 1e-12)$root
}

# Product B
adj_coef_prodB <- function(d, lambda = lambda_sc2, shape = shape_sc2, rate = rate_sc2, loading = theta,

  prem_net <- (1 + loading) * lambda * shape / rate -
    reinsurance_prem_rate("xl_prodB", d = d, lambda = lambda, xi = xi)

  lundberg <- function(r) lambda * (mgf_prodB(r, d, shape, rate) - 1) -   prem_net * r

  r_upper <- 0.1
```

```r
  while (lundberg(r_upper) <= 0) r_upper <- 2 * r_upper
  uniroot(lundberg, c(1e-8, r_upper), tol = 1e-12)$root
}

# Results
R_none <- adj_coef_none()

# Maximise R using Optimize for the Reinsurance Products
max_adjc_prodA <- optimize(adj_coef_prodA, c(minimum_alpha + 0.01, 1), maximum = TRUE)
max_adjc_prodB <- optimize(adj_coef_prodB, c(minimum_d + 0.05, 50),   maximum = TRUE)

q2c_adjc_results <- data.frame(
  Product  = c("No reinsurance", "Product A (optimal)", "Product B (optimal)"),
  Maximising_Parameter = c(NA, max_adjc_prodA$maximum, max_adjc_prodB$maximum),
  R         = c(adj_coef_none(), max_adjc_prodA$objective, max_adjc_prodB$objective))

q2c_adjc_results$lundberg_upper_bound <- exp(-q2c_adjc_results$R * c_0)
print(q2c_adjc_results)
```

```
##               Product Maximising_Parameter          R lundberg_upper_bound
## 1      No reinsurance                   NA 0.03034990            0.1883894
## 2 Product A (optimal)            0.6454339 0.03493003            0.1464382
## 3 Product B (optimal)            1.7173979 0.06598854            0.0265329
```

```r
### Q3 - Reinsurance Recommendations
exp_profit_rate_q3 <- function(type = "none", alpha = 1, d = Inf, lambda = lambda_sc2, EY = scen2_EY, l

  amount_ceded <- switch(type,
                         none       = 0,
                         prop_prodA = exp_claim_ceded_prodA(alpha),
                         xl_prodB   = exp_claim_ceded_prodB(d),
                        stop("type must be selected"))
  net_prem <- (1 + loading) * lambda * EY - reinsurance_prem_rate(type, alpha, d, lambda, xi)

  net_prem - lambda * (EY - amount_ceded)
}

# Expected Profit Results
exp_profit_prodA <- exp_profit_rate_q3(type = "prop_prodA", alpha = prodA_alpha)
exp_profit_prodB <- exp_profit_rate_q3(type = "xl_prodB", d = prodB_d)
q3_exp_profit_results <- rbind(exp_profit_prodA, exp_profit_prodB)

print(q3_exp_profit_results)
```

```
##                       [,1]
## exp_profit_prodA 0.1680000
## exp_profit_prodB 0.1679945
```

```r
# Recommendation for Reinsurance Boundary for Product A and B to Satisfy Condition 2
max_ceded_q3 <- 0.25 * theta * scen2_EY / reins_loading

#Product A
```

```r
alpha_profit_q3 <- 1 - max_ceded_q3 / scen2_EY

#Product B
d_profit_q3 <- uniroot(function(dd) exp_claim_ceded_prodB(dd) - max_ceded_q3, c(minimum_d, 50), tol = 1e

#Ruin Probabilities Under Updated Reinsurance Parameters
ruin_A_prof_limit <- est_ruin_prob_reins("prop_prodA", alpha = alpha_profit_q3)
ruin_B_prof_limit <- est_ruin_prob_reins("xl_prodB",   d = d_profit_q3)


#Expected Profit Under Recommendation
exp_profit_prodA_limit <- exp_profit_rate_q3(type = "prop_prodA", alpha = alpha_profit_q3)
exp_profit_prodB_limit <- exp_profit_rate_q3(type = "xl_prodB", d = d_profit_q3)

ruin_prob_q3_recommendation <- rbind(ruin_A_prof_limit, ruin_B_prof_limit, exp_profit_prodA_limit, exp_p

print(ruin_prob_q3_recommendation)
```

```
##                             psi           se       lower       upper
## ruin_A_prof_limit      0.01519 0.0003867721 0.014431927 0.015948073
## ruin_B_prof_limit      0.00308 0.0001752288 0.002736552 0.003423448
## exp_profit_prodA_limit 0.18000 0.1800000000 0.180000000 0.180000000
## exp_profit_prodB_limit 0.18000 0.1800000000 0.180000000 0.180000000
```

```r
#Results Table
exp_profit_none_q3 <- exp_profit_rate_q3(type = "none")

q3_results <- data.frame(
  contract   = c("No Reinsurance", "A (Specified)", "B (Specified)", "A (25% Profit Limit)", "B (25% Pro
  parameter  = c(NA, prodA_alpha, prodB_d, alpha_profit_q3, d_profit_q3),
  psi        = c(est_ruin_prod0["psi"],   est_ruin_prodA["psi"],   est_ruin_prodB["psi"], ruin_A_prof_li
  lower      = c(est_ruin_prod0["lower"], est_ruin_prodA["lower"], est_ruin_prodB["lower"], ruin_A_prof
  upper      = c(est_ruin_prod0["upper"], est_ruin_prodA["upper"], est_ruin_prodB["upper"], ruin_A_prof
  exp_profit = c(exp_profit_none_q3, exp_profit_prodA, exp_profit_prodB, exp_profit_prodA_limit, exp_pro

q3_results$profit_reduction_pct <- 100 * (1 - q3_results$exp_profit / exp_profit_none_q3)
q3_results$meets_survival_99    <- q3_results$psi <= 0.01
q3_results$meets_profit_25      <- q3_results$profit_reduction_pct <= 25 + 1e-8

print(q3_results)
```

```
##               contract parameter     psi       lower       upper exp_profit
## 1       No Reinsurance        NA 0.03269 0.031587836 0.033792164  0.2400000
## 2        A (Specified) 0.8000000 0.01266 0.011967043 0.013352957  0.1680000
## 3        B (Specified) 3.8890000 0.00206 0.001778977 0.002341023  0.1679945
## 4 A (25% Profit Limit) 0.8333333 0.01519 0.014431927 0.015948073  0.1800000
## 5 B (25% Profit Limit) 4.4021870 0.00308 0.002736552 0.003423448  0.1800000
##   profit_reduction_pct meets_survival_99 meets_profit_25
## 1              0.00000             FALSE            TRUE
## 2             30.00000             FALSE           FALSE
## 3             30.00231              TRUE           FALSE
## 4             25.00000             FALSE            TRUE
## 5             25.00000              TRUE            TRUE
```

```r
### Q4 Calibrating to Target Survival Probability
# Establish Parameters
target_psi_q4    <- 0.01
z_95_q4          <- qnorm(0.975)
target_half_q4   <- 0.0005
set.seed(676)

# Simulation Size: Smallest Number of Paths for 95% CI to be Narrower than Target Tolerance
n_required_q4    <- ceiling(z_95_q4^2 * target_psi_q4 * (1 - target_psi_q4) / target_half_q4^2)
n_sim_q4         <- ceiling(n_required_q4 / 50000) * 50000
ci_half_width_q4 <- z_95_q4 * sqrt(target_psi_q4 * (1 - target_psi_q4) / n_sim_q4)

q4_sim_size_results <- data.frame(n_required = n_required_q4, n_chosen = n_sim_q4, ci_half_width = ci_ha

print(q4_sim_size_results)
```

```
##   n_required n_chosen ci_half_width target_half
## 1     152122    2e+05  0.0004360645       5e-04
```

```r
# Ruin Probability Helper
est_ruin_prob_q4 <- function(type, value, n_sim) {

  if (type == "prop_prodA") est_ruin_prob_reins(type, alpha = value, n_sim = n_sim)
  else                      est_ruin_prob_reins(type, d = value, n_sim = n_sim)

}

# Product A
alpha_grid_q4 <- seq(0.60, 0.80, by = 0.01)
prodA_q4 <- data.frame(alpha = alpha_grid_q4, t(sapply(alpha_grid_q4, function(a) est_ruin_prob_q4("prop

print(prodA_q4)
```

```
##    alpha      psi           se       lower       upper
## 1   0.60 0.002280 0.0001066490 0.002070968 0.002489032
## 2   0.61 0.002710 0.0001162466 0.002482157 0.002937843
## 3   0.62 0.002810 0.0001183660 0.002578003 0.003041997
## 4   0.63 0.003320 0.0001286269 0.003067891 0.003572109
## 5   0.64 0.003410 0.0001303528 0.003154508 0.003665492
## 6   0.65 0.003770 0.0001370363 0.003501409 0.004038591
## 7   0.66 0.004625 0.0001517170 0.004327635 0.004922365
## 8   0.67 0.004920 0.0001564576 0.004613343 0.005226657
## 9   0.68 0.005270 0.0001618985 0.004952679 0.005587321
## 10  0.69 0.006265 0.0001764334 0.005919191 0.006610809
## 11  0.70 0.006000 0.0001726847 0.005661538 0.006338462
## 12  0.71 0.006485 0.0001794846 0.006133210 0.006836790
## 13  0.72 0.007365 0.0001911904 0.006990267 0.007739733
## 14  0.73 0.008120 0.0002006747 0.007726678 0.008513322
## 15  0.74 0.008415 0.0002042570 0.008014656 0.008815344
## 16  0.75 0.009400 0.0002157735 0.008977084 0.009822916
## 17  0.76 0.009830 0.0002206057 0.009397613 0.010262387
## 18  0.77 0.010555 0.0002285125 0.010107116 0.011002884
## 19  0.78 0.011490 0.0002383063 0.011022920 0.011957080
```

```
## 20  0.79 0.012055 0.0002440254 0.011576710 0.012533290
## 21  0.80 0.012295 0.0002464126 0.011812031 0.012777969
```

```
alpha_99_q4 <- prodA_q4$alpha[which.min(abs(prodA_q4$psi - target_psi_q4))]

# Product B
d_grid_q4 <- seq(4, 15, by = 0.5)
prodB_q4 <- data.frame(d = d_grid_q4, t(sapply(d_grid_q4, function(dd) est_ruin_prob_q4("xl_prodB", dd,
print(prodB_q4)
```

```
##       d      psi           se       lower       upper
## 1   4.0 0.002380 0.0001089572 0.002166444 0.002593556
## 2   4.5 0.003645 0.0001347537 0.003380883 0.003909117
## 3   5.0 0.005090 0.0001591240 0.004778117 0.005401883
## 4   5.5 0.007005 0.0001864930 0.006639474 0.007370526
## 5   6.0 0.008500 0.0002052773 0.008097657 0.008902343
## 6   6.5 0.010775 0.0002308560 0.010322522 0.011227478
## 7   7.0 0.012680 0.0002501921 0.012189623 0.013170377
## 8   7.5 0.013945 0.0002622073 0.013431074 0.014458926
## 9   8.0 0.015855 0.0002793172 0.015307538 0.016402462
## 10  8.5 0.017210 0.0002908076 0.016640017 0.017779983
## 11  9.0 0.018800 0.0003036985 0.018204751 0.019395249
## 12  9.5 0.020660 0.0003180658 0.020036591 0.021283409
## 13 10.0 0.021715 0.0003259099 0.021076217 0.022353783
## 14 10.5 0.023250 0.0003369676 0.022589543 0.023910457
## 15 11.0 0.023475 0.0003385552 0.022811432 0.024138568
## 16 11.5 0.024365 0.0003447560 0.023689278 0.025040722
## 17 12.0 0.025635 0.0003533967 0.024942342 0.026327658
## 18 12.5 0.025615 0.0003532625 0.024922606 0.026307394
## 19 13.0 0.027145 0.0003633741 0.026432787 0.027857213
## 20 13.5 0.028095 0.0003694974 0.027370785 0.028819215
## 21 14.0 0.028950 0.0003749126 0.028215171 0.029684829
## 22 14.5 0.028170 0.0003699760 0.027444847 0.028895153
## 23 15.0 0.029160 0.0003762293 0.028422591 0.029897409
```

```
d_99_q4 <- prodB_q4$d[which.min(abs(prodB_q4$psi - target_psi_q4))]

# Results
calibrated_prodA_q4 <- prodA_q4[prodA_q4$alpha == alpha_99_q4, ]
calibrated_prodB_q4 <- prodB_q4[prodB_q4$d == d_99_q4, ]

# Monte Carlo Sim Calibration Results
q4_calibration_results <- data.frame(
  contract   = c("A", "B"),
  parameter  = c(alpha_99_q4, d_99_q4),
  survival   = 1 - c(calibrated_prodA_q4$psi,   calibrated_prodB_q4$psi),
  surv_lower = 1 - c(calibrated_prodA_q4$upper, calibrated_prodB_q4$upper),
  surv_upper = 1 - c(calibrated_prodA_q4$lower, calibrated_prodB_q4$lower))

print(q4_calibration_results)
```

```
##   contract parameter survival surv_lower surv_upper
## 1        A      0.76 0.990170  0.9897376  0.9906024
## 2        B      6.50 0.989225  0.9887725  0.9896775
```

```r
# Acceptable Boundary
prodA_q4$overlaps_target <- (1 - prodA_q4$upper) <= 0.9905 & (1 - prodA_q4$lower) >= 0.9895
prodB_q4$overlaps_target <- (1 - prodB_q4$upper) <= 0.9905 & (1 - prodB_q4$lower) >= 0.9895

print(prodA_q4[prodA_q4$overlaps_target, ])
```

```
##    alpha      psi           se       lower       upper overlaps_target
## 16  0.75 0.009400 0.0002157735 0.008977084 0.009822916            TRUE
## 17  0.76 0.009830 0.0002206057 0.009397613 0.010262387            TRUE
## 18  0.77 0.010555 0.0002285125 0.010107116 0.011002884            TRUE
```

```r
print(prodB_q4[prodB_q4$overlaps_target, ])
```

```
##     d      psi          se      lower      upper overlaps_target
## 6 6.5 0.010775 0.000230856 0.01032252 0.01122748            TRUE
```

```r
### Q5 - Sensitivity Analysis
set.seed(676)

# Product B - Specified Retention (d = 3.889)
base_ruin_q5   <- est_ruin_prob_reins(type = "xl_prodB", d = prodB_d, n_sim = n_sim_q4)
base_profit_q5 <- exp_profit_rate_q3(type = "xl_prodB", d = prodB_d)

# Establish Cases
sensitivity_cases_q5 <- expand.grid(input  = c("c0", "d", "theta", "xi"),
                            change = c(-0.10, 0.10),
                            stringsAsFactors = FALSE)

# Run Sensitivity Analysis
sensitivity_analysis_q5 <- t(sapply(seq_len(nrow(sensitivity_cases_q5)), function(k) {

  inputs  <- list(c0 = c_0, d = prodB_d, theta = theta, xi = reins_loading)
  changed <- sensitivity_cases_q5$input[k]
  inputs[[changed]] <- inputs[[changed]] * (1 + sensitivity_cases_q5$change[k])

  ruin   <- est_ruin_prob_reins(type = "xl_prodB", d = inputs$d, c0 = inputs$c0, loading = inputs$theta

  profit <- exp_profit_rate_q3(type = "xl_prodB", d = inputs$d, loading = inputs$theta, xi = inputs$xi)

  c(ruin, exp_profit = profit)
}))

# Results
q5_sensitivity_results <- data.frame(sensitivity_cases_q5, sensitivity_analysis_q5)
q5_sensitivity_results$pct_change_psi    <- 100 * (q5_sensitivity_results$psi / base_ruin_q5["psi"] - 1
q5_sensitivity_results$pct_change_profit <- 100 * (q5_sensitivity_results$exp_profit / base_profit_q5 -

print(q5_sensitivity_results)
```

```
##   input change      psi           se        lower        upper exp_profit
## 1    c0   -0.1 0.004440 1.486655e-04 0.0041486156 0.0047313844  0.1679945
## 2     d   -0.1 0.001020 7.137785e-05 0.0008800994 0.0011599006  0.1571526
```

```
## 3 theta   -0.1 0.002480 1.112171e-04 0.0022620144 0.0026979856  0.1439945
## 4    xi   -0.1 0.001840 9.582835e-05 0.0016521764 0.0020278236  0.1751950
## 5    c0    0.1 0.000785 6.262523e-05 0.0006622546 0.0009077454  0.1679945
## 6     d    0.1 0.003080 1.239055e-04 0.0028371453 0.0033228547  0.1773062
## 7 theta    0.1 0.001710 9.238712e-05 0.0015289212 0.0018910788  0.1919945
## 8    xi    0.1 0.002305 1.072308e-04 0.0020948277 0.0025151723  0.1607939
##   pct_change_psi pct_change_profit
## 1     130.649351          0.000000
## 2     -47.012987         -6.453674
## 3      28.831169        -14.286186
## 4      -4.415584          4.286186
## 5     -59.220779          0.000000
## 6      60.000000          5.542879
## 7     -11.168831         14.286186
## 8      19.740260         -4.286186
```